CalorieSense

AI-Powered Calorie Prediction & Adaptive Fitness Coaching

Final Project Presentation

TEAM MEMBERS

D.M.I.N Dissanayake	KIC-HNDCSAI252F-003
C.A Aneek Ahamed	KIC-HNDCSAI252F-012
M.K Imadh Ahamed	KIC-HNDCSAI252F-032
M.H. Nasrullah	KIC-HNDCSAI252F-031
M.M Abdullah	KIC-HNDCSAI252F-039

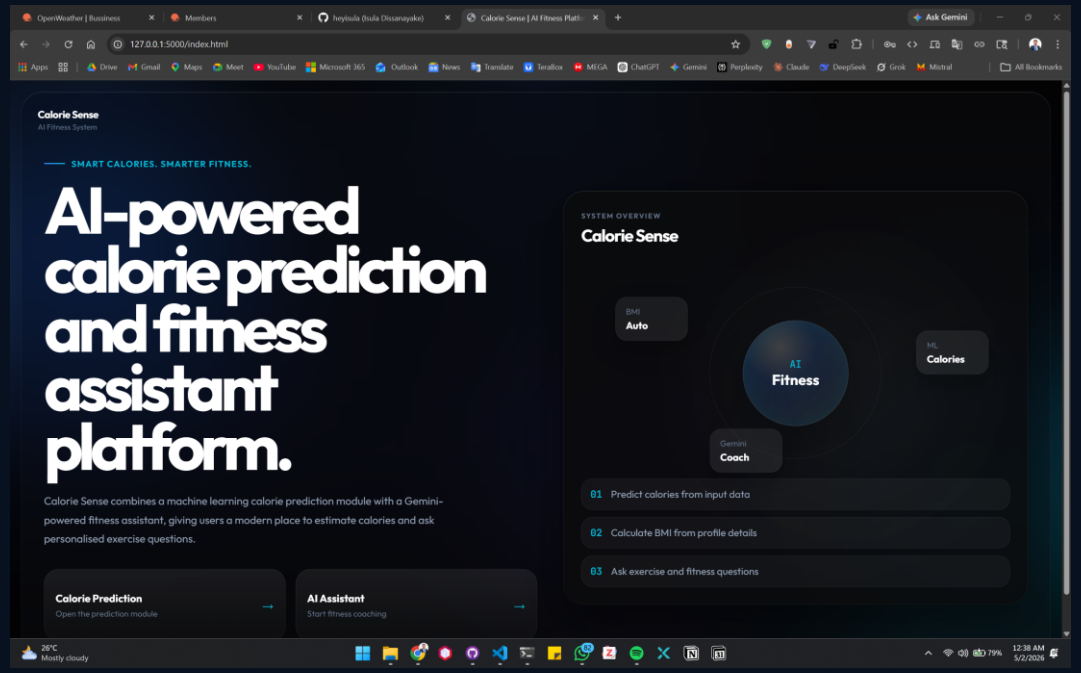
Final Project Presentation

The gap CalorieSense solves

WHY THIS PROJECT MATTERS

Many fitness tools still estimate calories using generic formulas. CalorieSense was built to make calorie prediction more personalised, context-aware, and useful in real fitness sessions.

- Traditional estimators often ignore age, weight, gender and real exercise intensity.
- Weather and environment are normally not part of calorie estimation.
- Most apps stop at numbers; users still need practical exercise guidance.



Project objectives

WHAT WE SET OUT TO BUILD

01 Predict calories

Train a model to estimate calories burned from workout and body inputs.

02 Add weather awareness

Use live weather as extra context during prediction.

03 Compare models

Evaluate Ridge, Random Forest, XGBoost and ANN fairly.

04 Deploy as an app

Build a Flask web app with prediction and AI coach modules.

05 Coach users

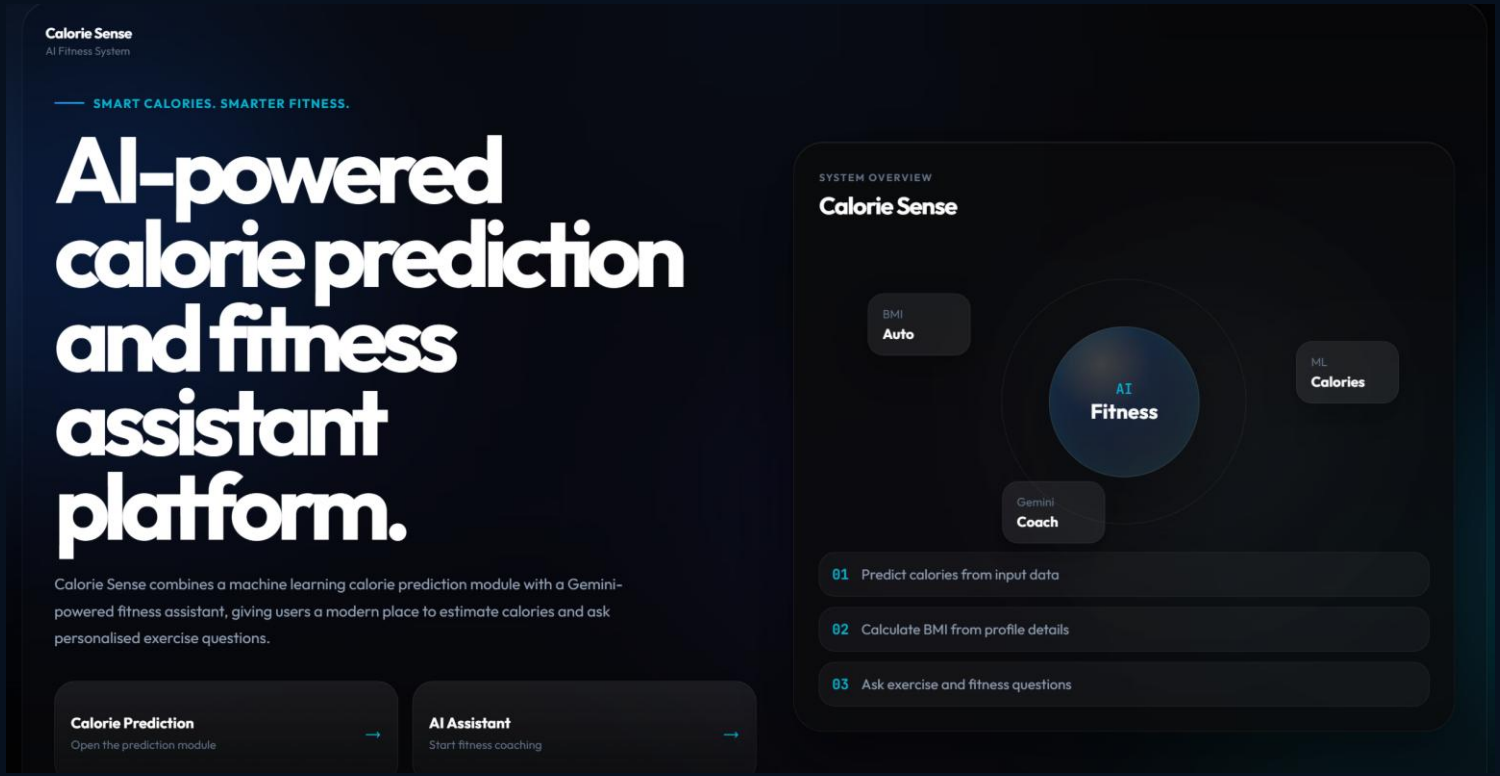
Use Gemini with fitness datasets to answer exercise questions.

06 Handle data issues

Diagnose dataset problems and justify final decisions.

System overview

THREE MODULES WORKING TOGETHER



CORE IDEA

A single web platform combines machine learning prediction, real-time weather context and Gemini-powered fitness coaching.

- ML calorie prediction
- Weather API
- Gemini coach
- BMI profile

Dataset strategy

WHAT EACH DATA SOURCE DOES

1	**Exercise & Fitness Metrics**	ANN training data
2	**ExerciseDB**	Exercise library for Coaching and Recommendation
3	**Mendeley Gym Recommendation**	Exercise library for Coaching and Recommendation
4	**OpenWeatherMap API**	Live weather context

Model Training Dataset: Dataset 2 only, because the planned merge created incompatible calorie distributions and many missing values.

3,864
training rows

4
data sources

80/20
train-test split

A hidden data problem was discovered

THE ORIGINAL TARGET COULD NOT BE LEARNED

Early models failed because the original Calories Burn column behaved like random synthetic noise.

- Feature correlations were almost zero before fixing the target.
- KS test showed the target behaved like a uniform random distribution.
- This explained the early negative or near-zero R^2 results.

Before fix	After fix
Duration +0.02	Duration +0.67
Heart Rate −0.04	Heart Rate +0.59
Intensity +0.01	Intensity +0.18

The target was rebuilt scientifically

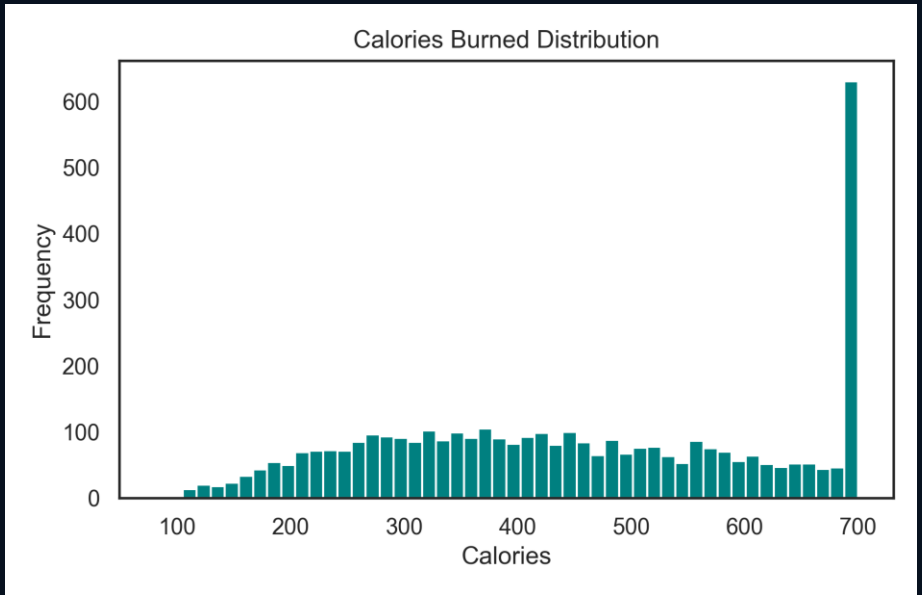
KEYTEL FORMULA + REALISTIC ADJUSTMENTS

Instead of accepting a random target, the project rebuilt calories using a validated heart-rate based energy expenditure equation.

Male / Female formula uses:

- Duration
- Heart Rate
- Weight
- Age
- Gender

+ intensity adjustment + realistic Gaussian noise



Critical fix: Calories column was rebuilt

THE ORIGINAL TARGET COULD NOT BE LEARNED

01 Problem found

The original Calories Burn column behaved like random synthetic noise. Early models produced near-zero / negative R^2 because there was no meaningful relationship between inputs and target.

02 Scientific replacement

We removed the unreliable target and rebuilt calories using the validated Keytel et al. heart-rate formula using duration, heart rate, weight, age and gender.

03 Final result

The rebuilt target created real physiological signal. Duration, heart rate and intensity became meaningful predictors, allowing the model to learn properly.

Male/Female Keytel formula → intensity adjustment → weather multiplier → final Calories Burned target

Weather-aware calorie prediction

ADDING REAL-WORLD CONTEXT

Weather is fetched live and used as a conservative physiological adjustment at inference time.

Sunny

+5%

heat dissipation strain

Cloudy

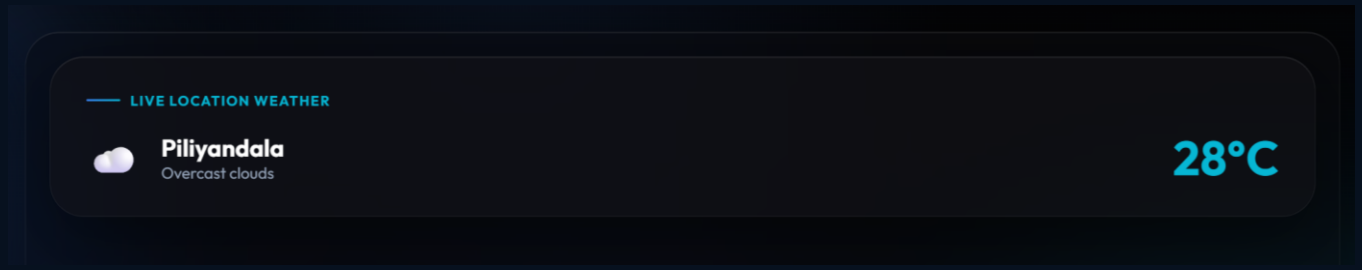
0%

baseline condition

Rainy

+8%

cold-induced thermogenesis



Leakage-free preprocessing pipeline

FROM RAW DATA TO MODEL INPUTS

01 Load raw data

02 Drop non-predictive columns

03 Convert duration

04 Rebuild target

05 Encode gender + weather

06 Engineer features

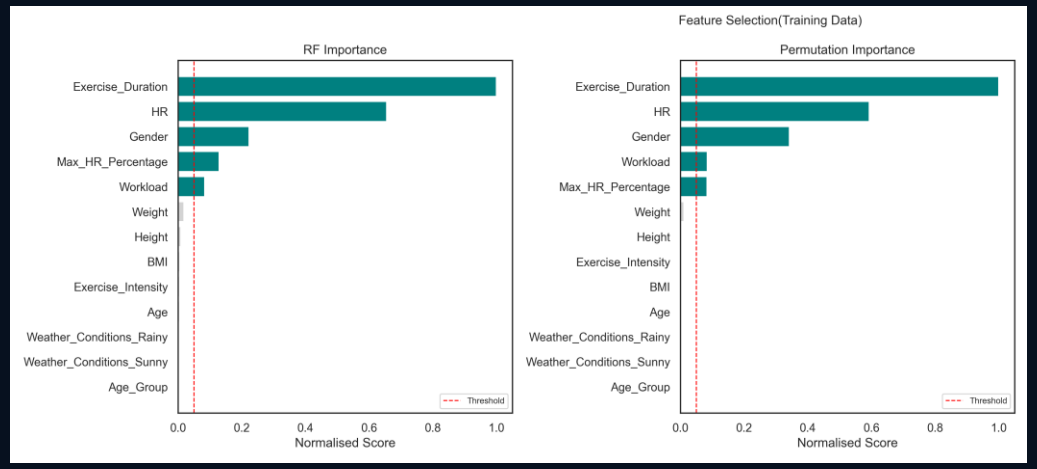
07 Train/test split

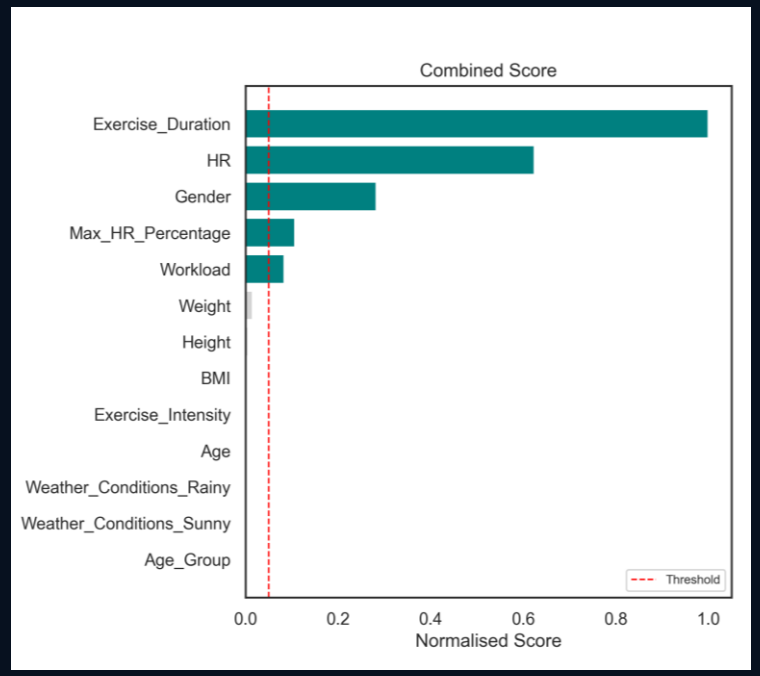
08 Feature selection on train only

09 Scale train then test

Feature selection confirmed the strongest signals

DURATION AND HEART RATE DOMINATE





Selected 7 features from 13 candidates using Random Forest importance and Permutation Importance on training data only.

- Exercise Duration was the strongest feature.
- Heart Rate was the second strongest signal.
- Weather has low importance by design, but still affects live inference.

Model development journey

THREE NOTEBOOK VERSIONS, ONE FINAL PIPELINE

V1

Merged two datasets

Failed due to incompatible distributions and many nulls.

V2

Single dataset

Merge issue solved, but random target still caused poor learning.

V3

Final production pipeline

Target rebuilt, weather added, leakage fixed, model deployed.

Models trained and evaluated

SIX FAIR COMPARISONS ON THE SAME FINAL PIPELINE

Baseline

Ridge Regression

Polynomial interaction baseline for simple explainable comparison.

Untuned

Random Forest

Tree ensemble model trained before tuning.

Tuned

Random Forest tuned

RandomizedSearchCV tuning with 5-fold validation.

Untuned

XGBoost

Gradient boosting model trained before tuning.

Tuned

XGBoost tuned

Strongest competitor, but slightly larger generalisation gap.

Selected

ANN

Artificial Neural Network with dense layers, BatchNorm, Dropout and Huber loss.

All models were tested after the target rebuild, feature engineering, feature selection and scaling steps.

ANN architecture

FINAL PRODUCTION MODEL

The production model is a regularised neural network designed for stable regression and strong generalisation.

Input: 7 selected features

Dense 256 + LeakyReLU + BatchNorm + Dropout

Dense 128 + LeakyReLU + BatchNorm + Dropout

Dense 64 + LeakyReLU + BatchNorm + Dropout

Dense 32 + LeakyReLU

Dense 1 output

Adam optimizer

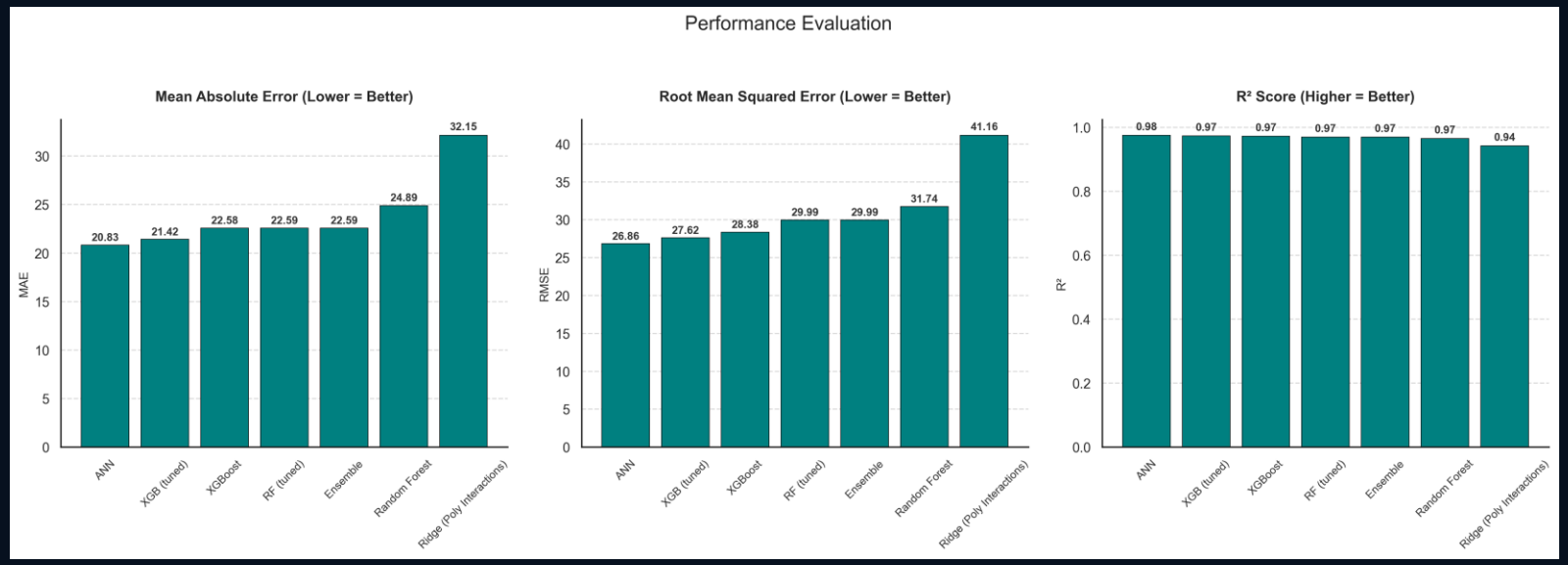
Huber loss

Early Stop regularisation

Reduce LR callback

Model comparison results

ANN SELECTED FOR BEST GENERALISATION



0.9757
ANN Test R^2

20.83
ANN Test MAE

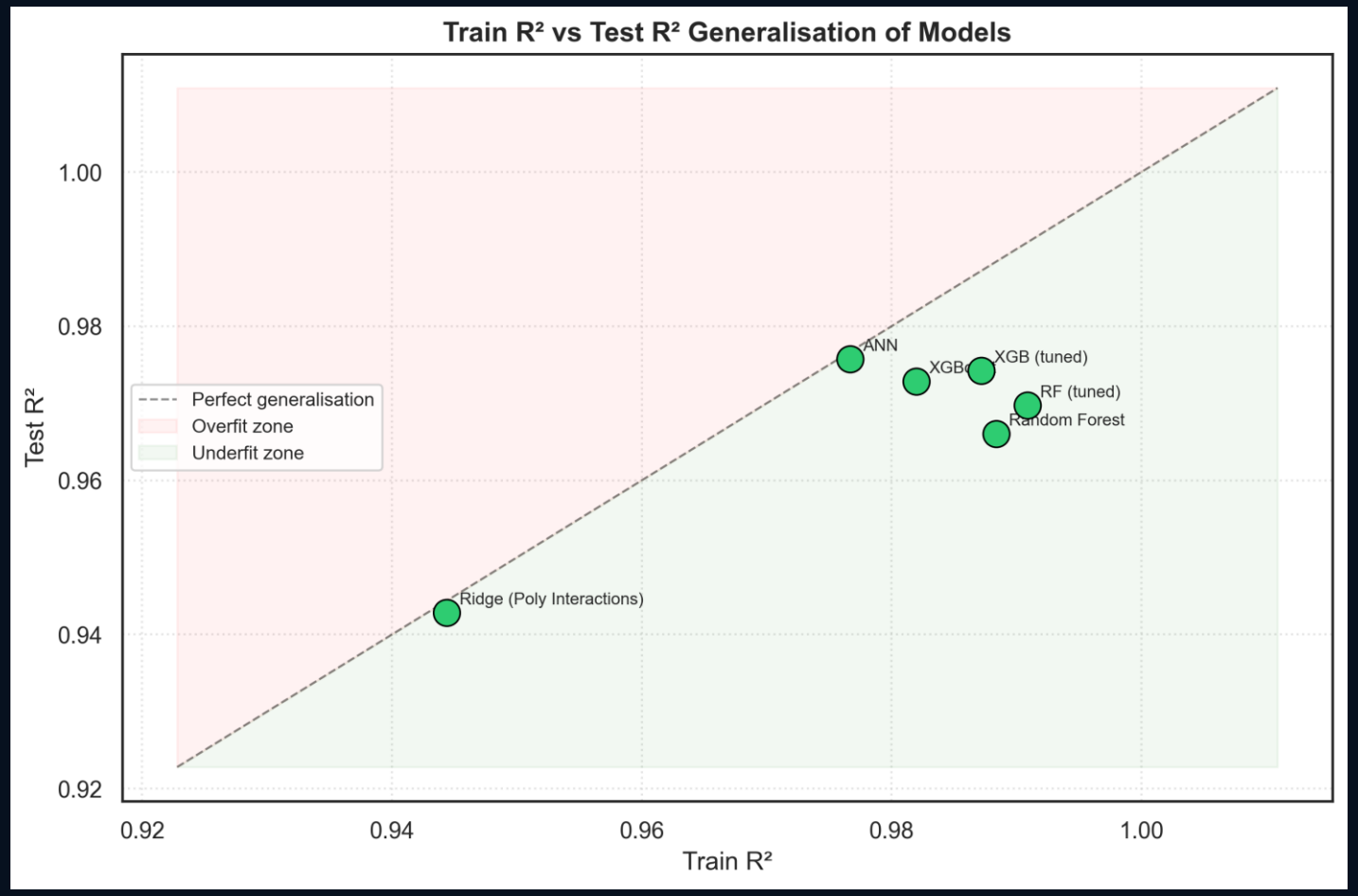
0.0011
Train/Test gap

26.86
Test RMSE

The ANN produced the most balanced result: high accuracy with the smallest generalisation gap across all tested models.

Why ANN over tuned XGBoost?

GENERALISATION MATTERED MORE THAN TRAINING SCORE



XGBoost tuned had slightly stronger training R^2, but the ANN had a much smaller train/test gap. For live predictions, stable performance on unseen users is more important.

0.0130
XGB tuned ΔR^2

0.0011
ANN ΔR^2

Selected model: Artificial Neural Network

CHOSEN FOR STRONGEST GENERALISATION IN LIVE PREDICTION

0.9757
Test R^2

20.83 kcal
Test MAE

0.0011
ΔR^2 gap

A Why not only choose the highest train score?

Tuned XGBoost had a slightly higher training R^2, but its train/test gap was much larger. For real users, every input is unseen data, so stable generalisation is more important than memorising training patterns.

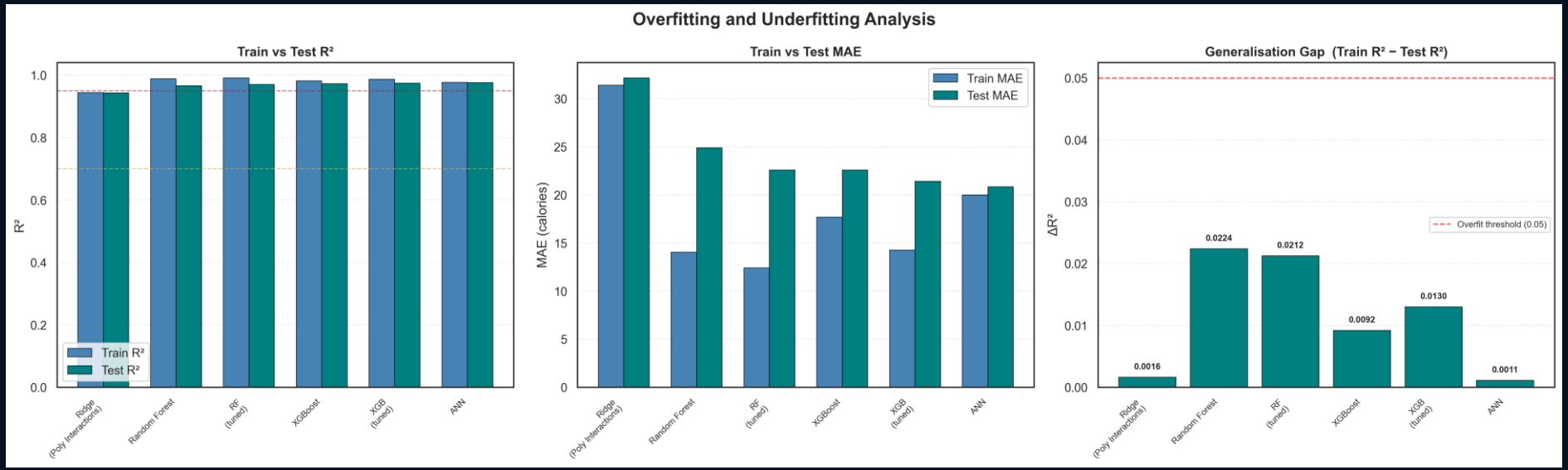
B Why ANN was the final production model

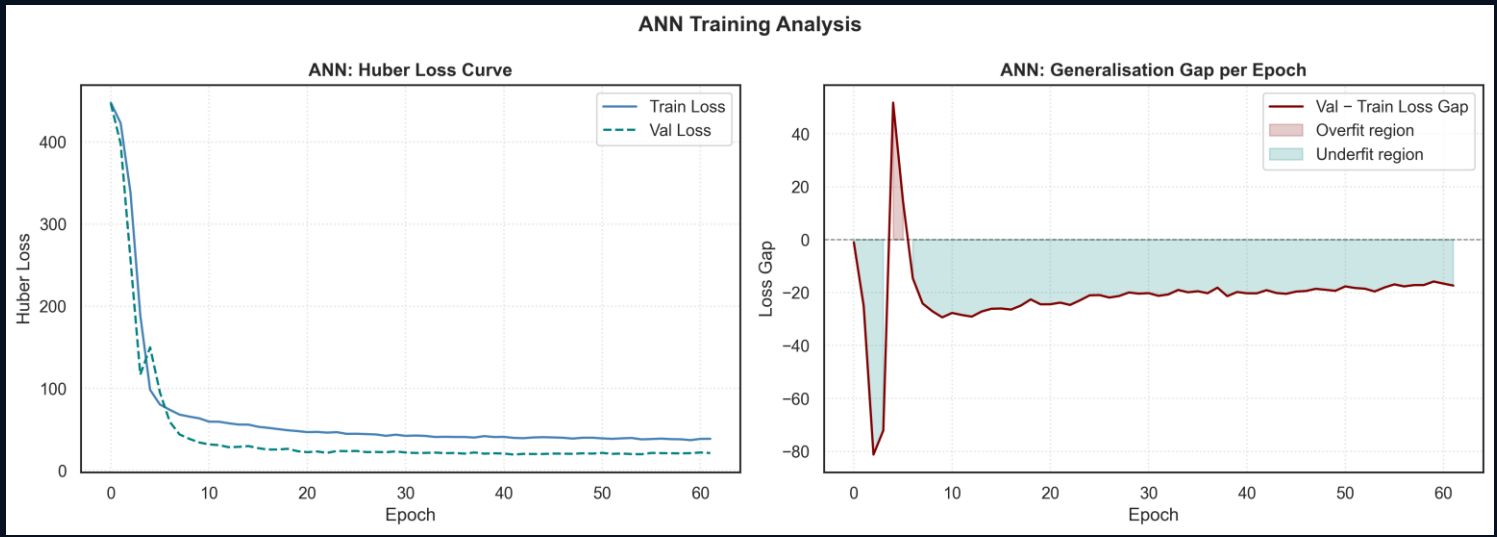
The ANN delivered almost the same high accuracy while keeping the smallest train/test gap. This made it the safest option for deployment inside the Flask prediction API.

Final decision: ANN gave high accuracy with the smallest generalisation gap.

ANN training diagnostics

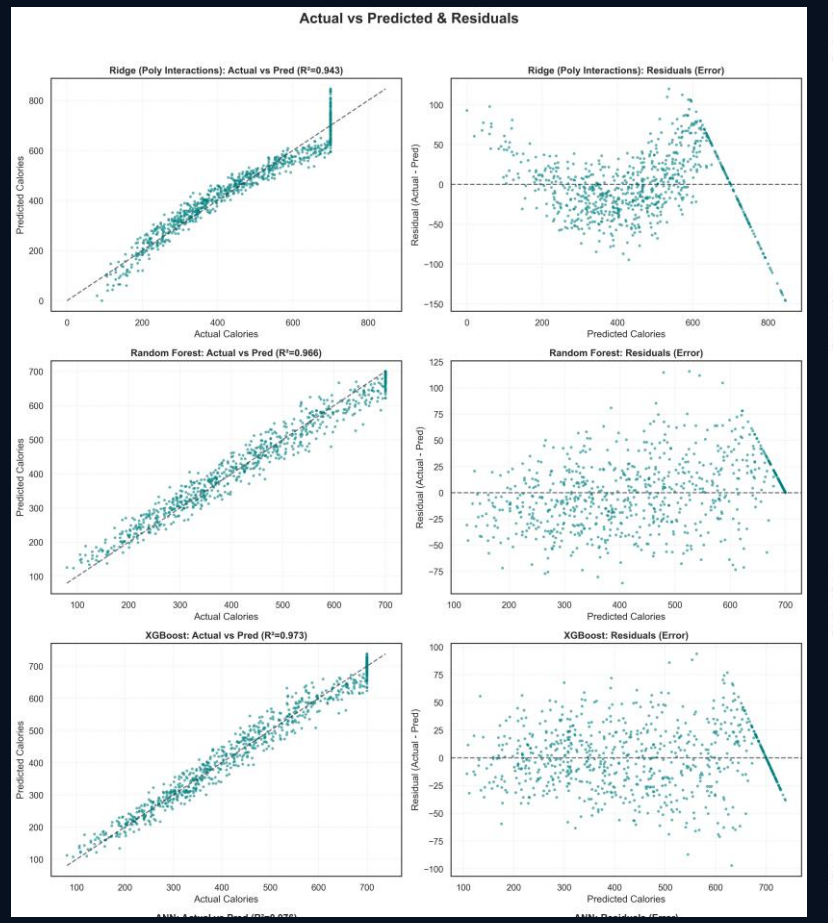
CLEAN CONVERGENCE WITHOUT OVERFITTING

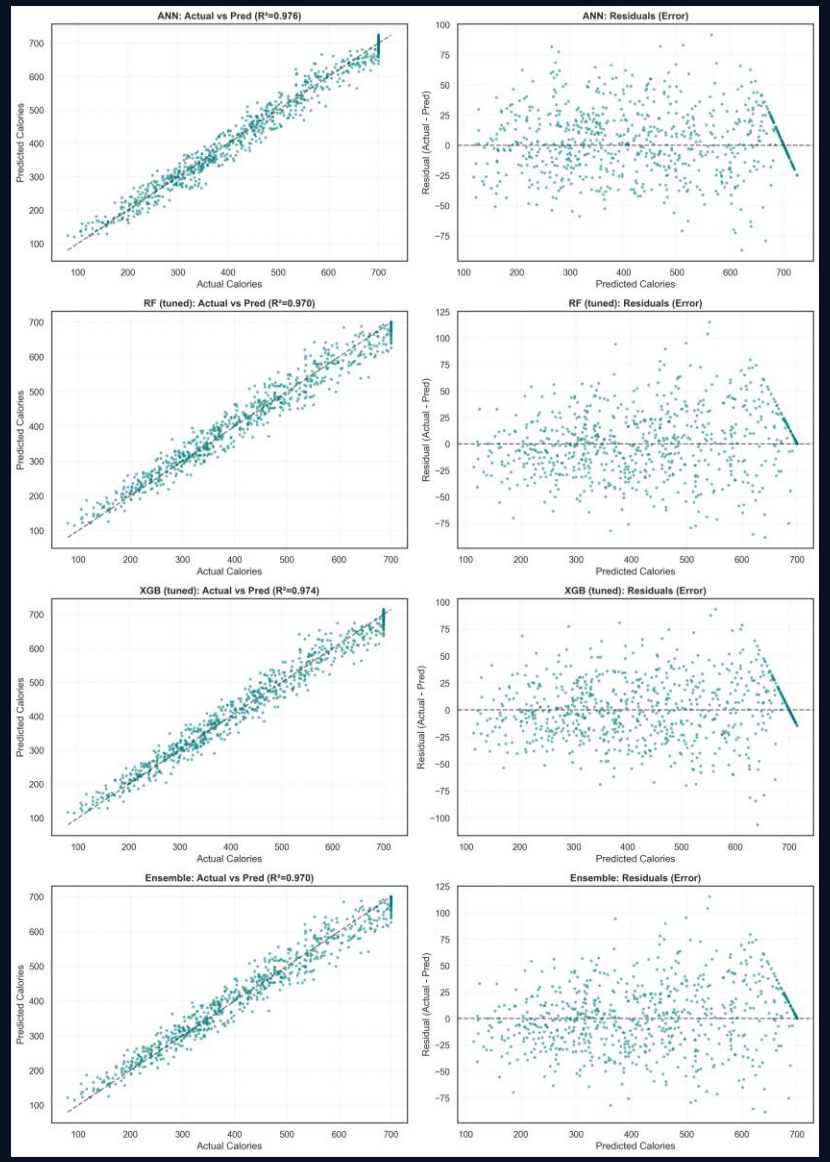




Prediction quality

ACTUAL VS PREDICTED CALORIES



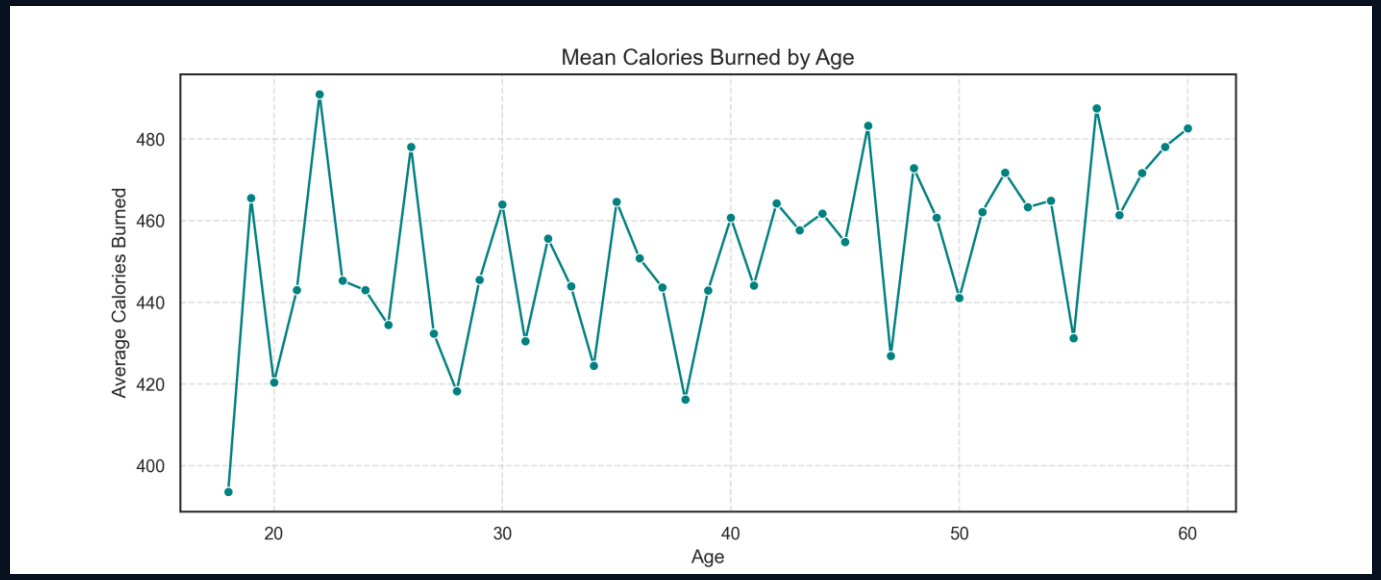


Predictions stay close to the ideal line, and residuals are centred near zero. This supports the claim that the ANN is not systematically over- or under-estimating.

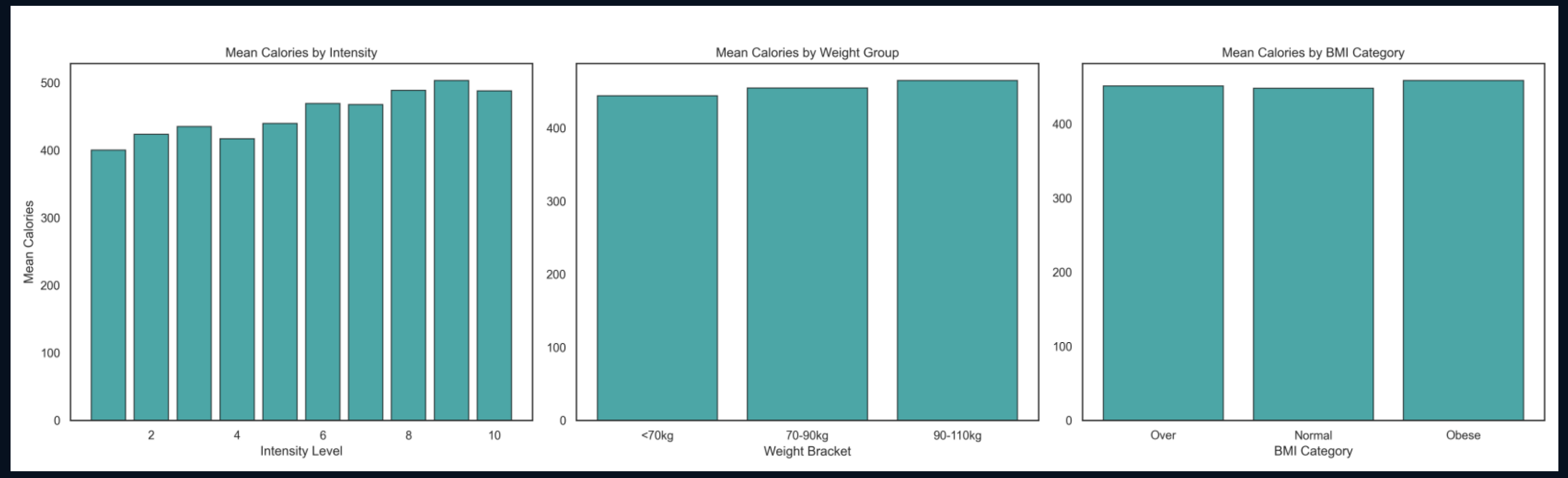
- Good alignment between actual and predicted values.
- Residuals are approximately balanced around zero.
- Model behaviour is explainable and deployment-ready.

Exploratory data analysis highlights

DATA BEHAVIOUR AFTER THE FIX

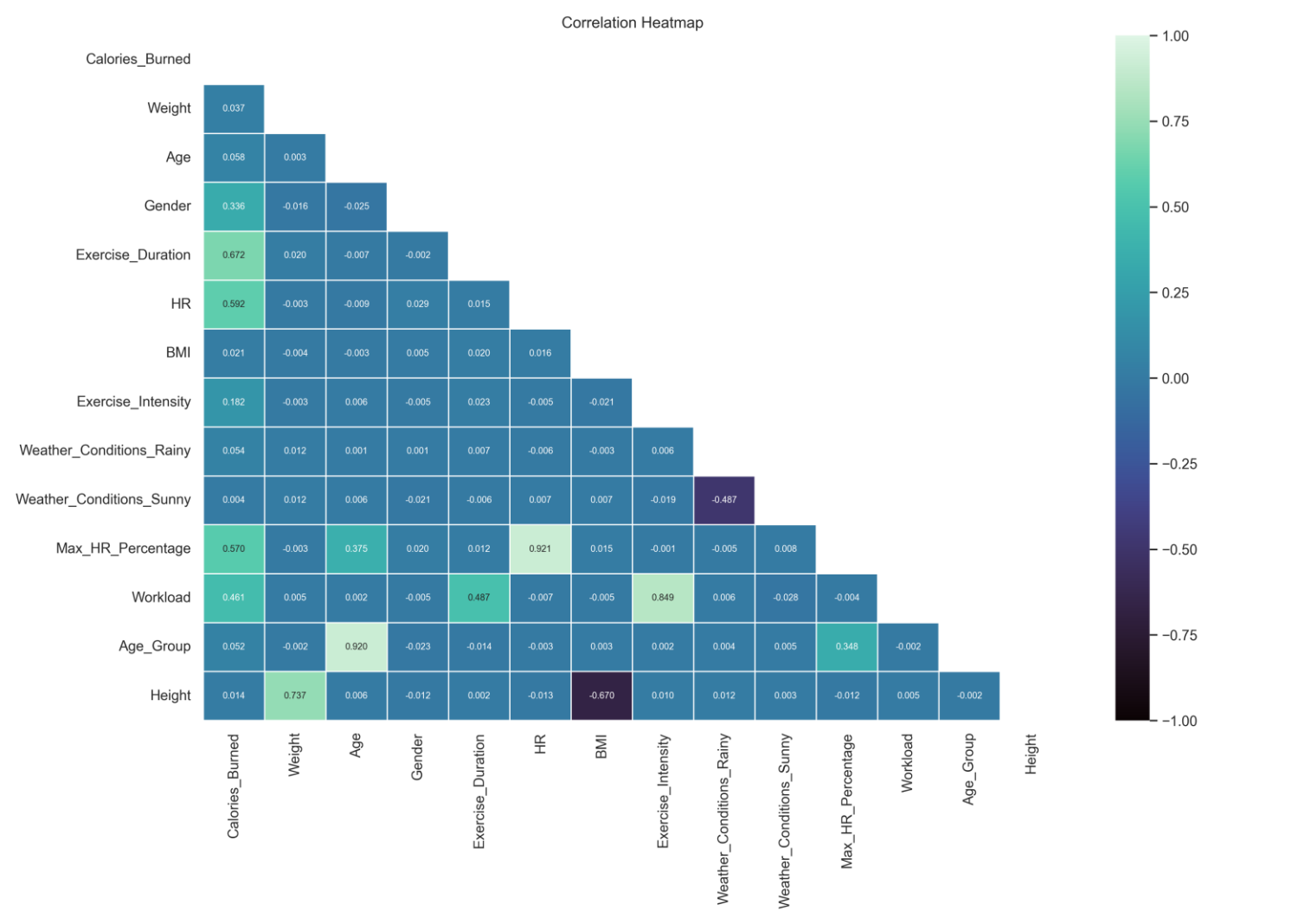


EDA confirmed the target became meaningful after the Keytel replacement. Duration and heart rate became the strongest practical indicators of calorie burn.



Exploratory data analysis highlights

DATA BEHAVIOUR AFTER THE FIX



EDA confirmed the target became meaningful after the Keytel replacement. Duration and heart rate became the strongest practical indicators of calorie burn.

System architecture

FLASK BACKEND + ML ARTIFACTS + API LAYERS

User Browser

HTML/CSS/JS interface

Flask app.py

Routes requests and loads models

/api/weather

IP location → OpenWeatherMap

/api/predict_calorie

Scaler → ANN → kcal output

/api/start

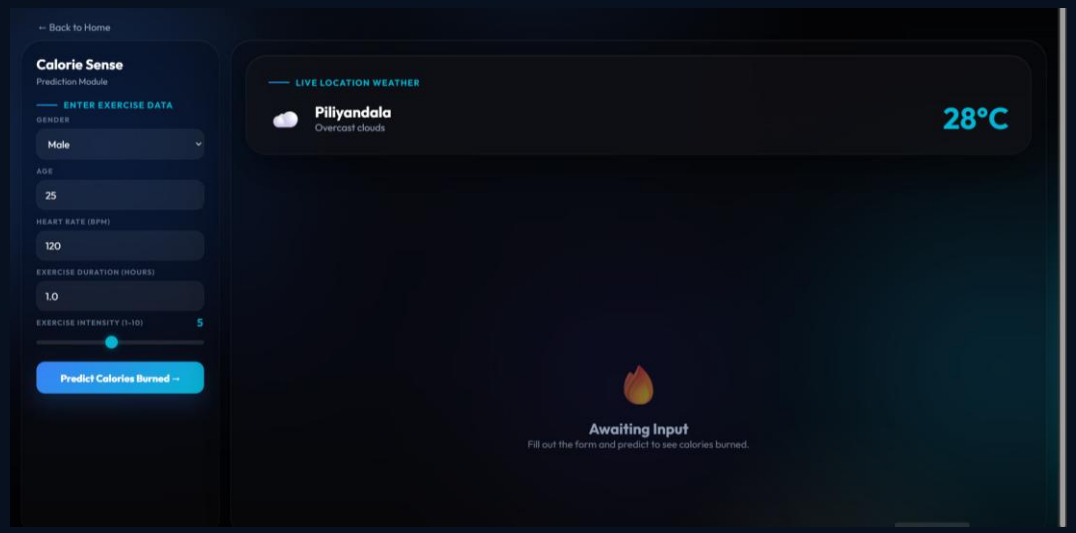
Profile → BMI → lookup match

/api/chat

Gemini with grounded fitness prompt

Calorie prediction module

FROM FORM INPUT TO KCAL OUTPUT



1. User enters workout details

2. Weather is fetched automatically

3. Derived features are computed

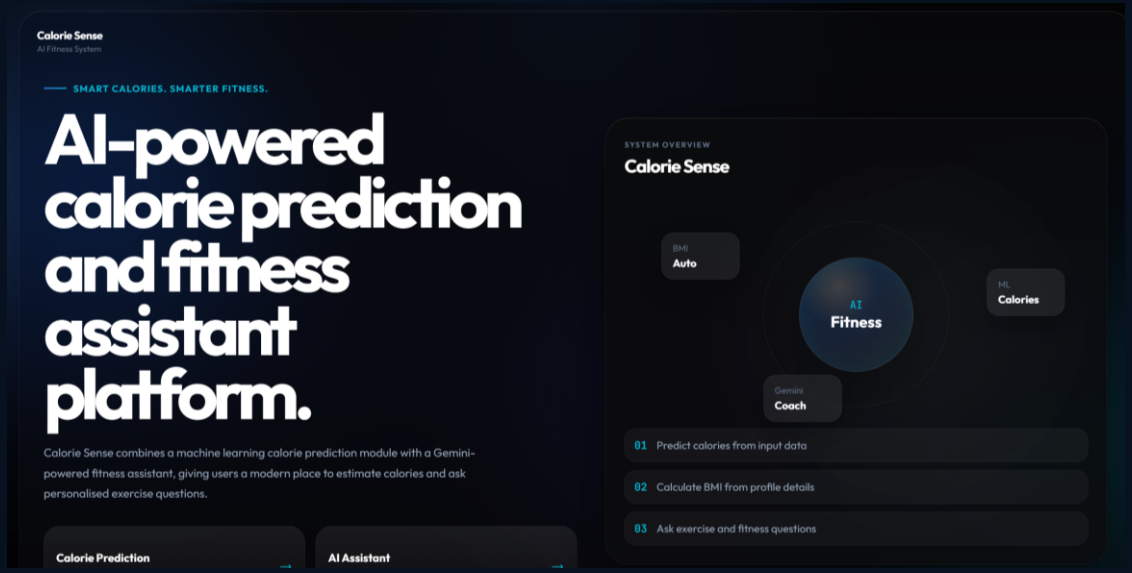
4. Scaler prepares the feature vector

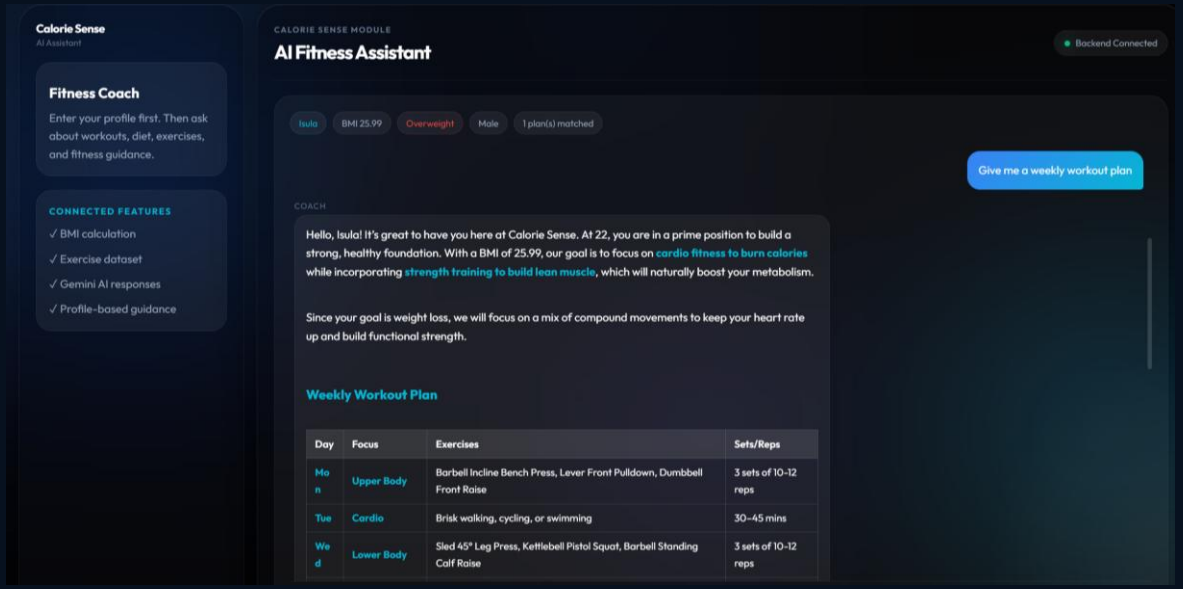
5. ANN predicts calorie expenditure

6. Result appears in the same page

Web application interface

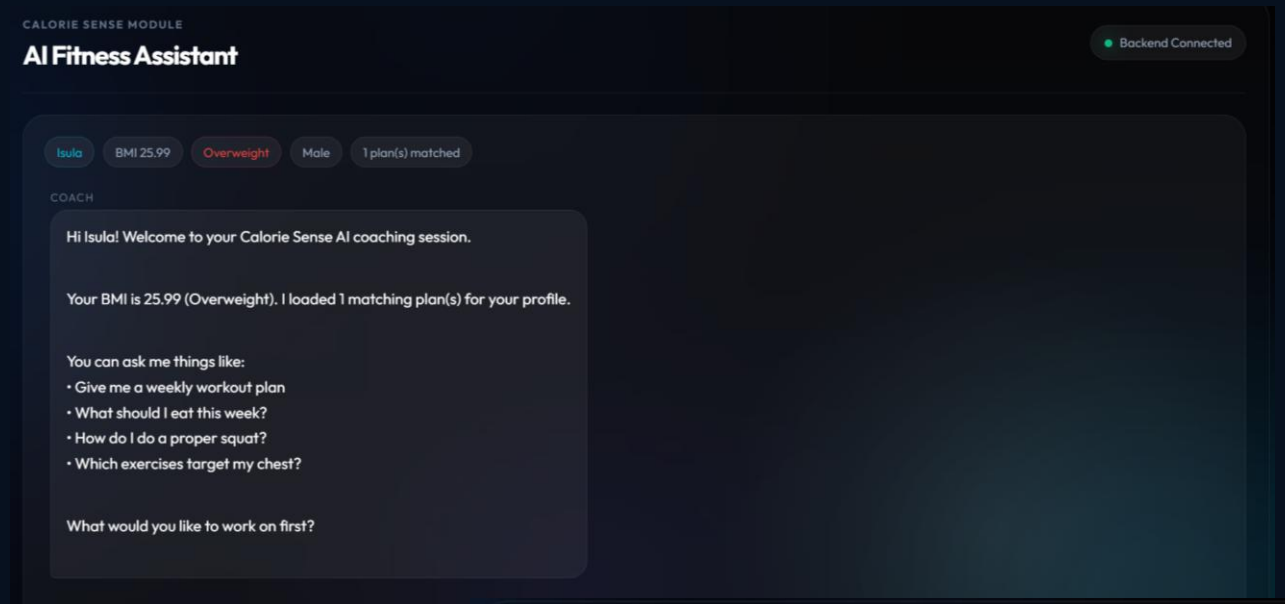
DARK, MODERN AND PRESENTATION-READY

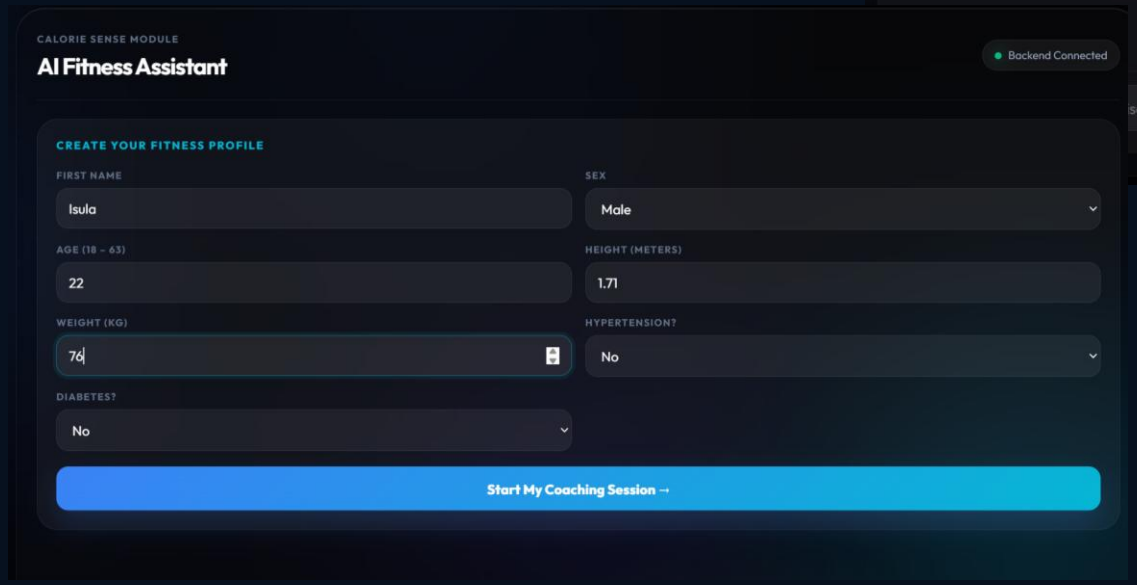


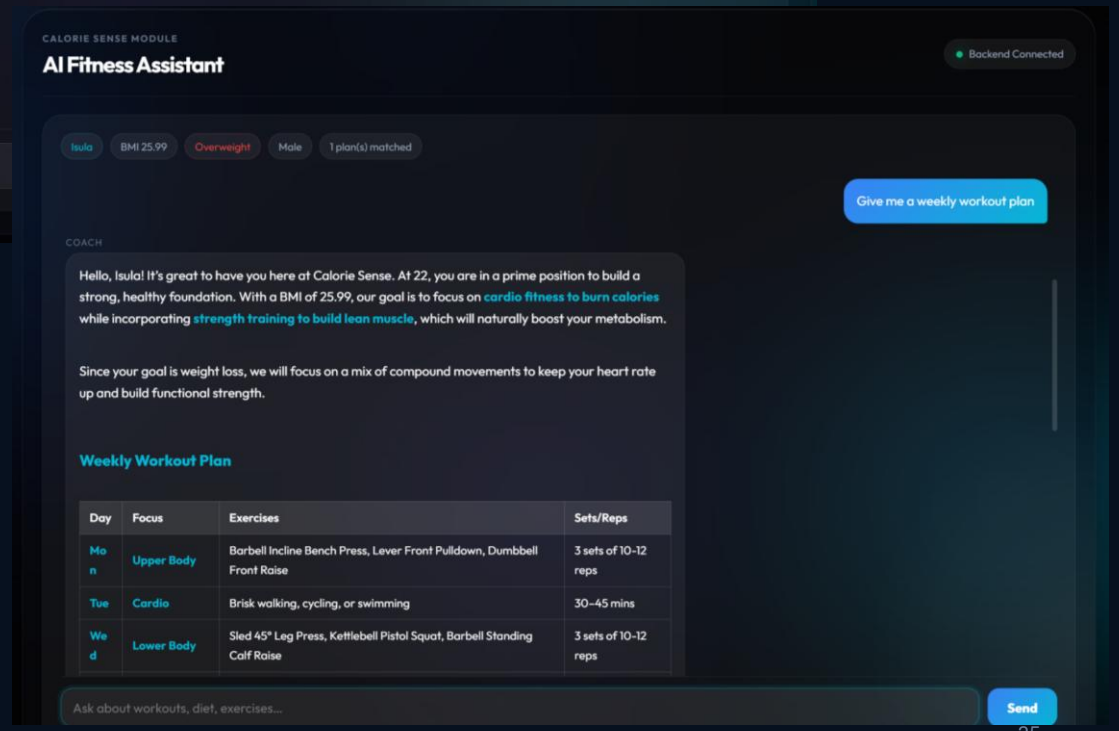


AI fitness coach module

PROFILE-BASED GUIDANCE WITH GEMINI







Key challenges faced

WHAT MADE THE PROJECT REALISTIC

Random target	The original Calories Burn column had no learning signal.
Dataset merge issue	Two calorie distributions and many null values made merging unreliable.
Leakage prevention	Pipeline order had to be corrected across iterations.
Weather importance	Low feature score needed careful scientific explanation.
API reliability	Weather and Gemini integration required fallback handling.

Limitations and future improvements

HONEST NEXT STEPS

Current limitations

- Formula-derived target is not direct oxygen-consumption measurement.
- Dataset size is moderate for edge cases.
- Gemini and weather APIs require internet and active keys.
- No long-term user history tracking yet.

Future improvements

- Collect real wearable or lab-measured calorie data.
- Add user history and progressive personalisation.
- Build a mobile app and wearable integrations.
- Use richer weather features: temperature, humidity and wind.

Conclusion

FINAL MESSAGE

CalorieSense is more than a prediction demo. It shows a full AI development journey: finding a broken target, fixing it with scientific reasoning, comparing models fairly, and deploying a working fitness platform.

ANN production model

95.38% Prediction Accuracy

Gemini AI coach

Weather context-aware

Thank you